LSUI Dataset Exploration: Cell-by-Cell Report

Notebook: `01_lsui_dataset_exploration.ipynb` **Dataset:** LSUI (Large-Scale Underwater Image), 4,279 paired images (raw input and enhanced ground truth) **Goal:** understand the data well enough to train an underwater image enhancement (UIE) model such as WaterMamba, and to know what baseline a trained model must beat.

For each cell this report gives four things: **What the code does**, **Output**, **Why we do it**, and **What the result tells us**.

Big picture

Before training any model, we need answers to five questions:

1. Is the data complete and correctly paired? (Cells 1-2)
2. What sizes and shapes are the images? (Cells 3-5)
3. What does the degradation look like, and what did the ground truth fix? (Cells 6-11)
4. What score does doing nothing get? (Cell 12)
5. How do we split the data reproducibly? (Cell 13)

Cell 0: Environment setup

What the code does

- Imports `os, sys, time, random, pathlib, numpy, pandas, matplotlib, seaborn, cv2` (OpenCV), `PIL`, and `tqdm`.
- Falls back to `print` if IPython's `display` is unavailable.
- Sets plot sizes, font sizes, and the seaborn theme.
- Sets `RANDOM_SEED = 42` for both `random` and `numpy`.

Output: a success message with NumPy 2.5.2, Pandas 2.3.3, and OpenCV 5.0.0.

Why we do it

- Fixed seeds make every random sample (cells 7, 9, 11, 12, 13) the same on each run. Results can be reproduced and compared later.
- Printing versions records the environment. Library-version differences are a common cause of "it worked yesterday" bugs.

What the result tells us: the environment loaded without errors. Nothing about the data yet.

Cell 1: Locate the dataset

What the code does

- Resolves the current directory and checks whether `dataset/LSUI` exists there or in the parent folder.
- Sets `PROJECT_ROOT`, `DATASET_DIR`, `INPUT_DIR` (raw images), and `GT_DIR` (ground truth).
- Raises `FileNotFoundError` if neither location works.

Output: root is `E:\sem5_project`, and both `input` and `GT` folders exist (`True`).

Why we do it: this makes the notebook portable. It runs from the project root or from a `notebooks/` subfolder

without editing paths. Failing early with a clear message is better than a confusing error later.

What the result tells us: the dataset is on disk in the expected structure (`input/` and `GT/`).

Cell 2: Verify one-to-one pairing

What the code does

- Lists all `.jpg` files in both folders, sorted numerically by filename stem so `2.jpg` comes before `10.jpg` .
- Uses set operations to compute: files common to both, files missing in GT, and files missing in input.
- `assert` stops the notebook if the three counts disagree.

Output

Check	Result
Input images	4,279
GT images	4,279
Matching pairs	4,279
Missing in GT / Input	0 / 0

Why we do it: supervised enhancement trains on pairs, where the network learns a mapping from `input/N.jpg` to `GT/N.jpg` . One missing or misnamed file would silently pair the wrong images, or crash training hours in. Verifying this first is cheap insurance.

What the result tells us: the dataset is complete, with 4,279 valid training pairs. Note that this is slightly fewer than the 4,500 + 504 pairs described in the LSUI/WaterMamba papers, so published splits cannot be reused as-is.

Cell 3: Build the metadata table

What the code does

- Loops over all 4,279 pairs.
- Records width, height, aspect ratio (width/height), megapixels, input file size (KB), GT file size (KB), and color mode.
- Reads size and mode with PIL, which reads only the image header and does not decode the pixels. This is why 4,279 images take under a second.
- Stores everything in the DataFrame `df_meta` and shows the first five rows.

Output: 4,279 records. For example, `0.jpg` is 640×320, a 2.0 aspect ratio and 0.205 MP, with a 18.8 KB input and a 34.2 KB GT, in RGB mode.

Why we do it: the table is the foundation for everything that follows. It is used for the statistics and plots in cells 4-5, for random sampling in cells 7-12, and for the split in cell 13. Checking `mode` ensures there are no grayscale or RGBA images that would break a 3-channel network.

What the result tells us: the images are RGB and have different sizes (even in the first five rows), so resolution will need handling.

Cell 4: Summary statistics

What the code does: runs `.describe()` on width, height, aspect ratio, megapixels, and both file sizes, then transposes it.

Output (key rows)

Metric	Mean	Min	Median	Max
Width	447	256	320	1280
Height	309	225	256	1024
Aspect ratio	1.42	1.0	1.333	2.0
Megapixels	0.163	0.066	0.077	1.311
Input KB	49.5	6.0	27.3	690.8
GT KB	65.1	10.4	43.7	721.1

Why we do it: numbers give precise answers where plots give impressions. They show the typical image and the extremes.

What the result tells us

- The median image is tiny (about 320×256), while the largest is about 17× bigger in pixels. The distribution is skewed, with a few large images pulling the mean above the median.
- A fixed training size such as 256×256 is reasonable, since most images are already near that size.
- GT files are larger than their inputs on average (65 vs 49 KB). JPEG size tracks detail and texture, so the GT contains more high-frequency information. Cell 11 tests this directly.

Cell 5: Four overview plots

What the code does: draws a 2×2 figure.

1. Scatter of width vs. height, showing which resolutions are common.
2. Histogram with KDE of megapixels.
3. Bar chart of the six most common aspect ratios, with counts on the bars.
4. Overlaid KDE of input and GT file sizes.

Output: the figure plus a harmless seaborn `FutureWarning` about `palette` without `hue`.

Why we do it: plots show structure that statistics hide, such as clusters, multimodality, and outliers.

What the result should tell us

- *Scatter and aspect-ratio bars:* images likely come in a few standard formats (for example 4:3 and 16:9) rather than at random. This informs whether to resize, center-crop, or random-crop.
- *Megapixels:* a right-skewed shape confirms most images are small, with a long tail of large ones.
- *File-size KDE:* the GT curve sitting to the right of the input curve supports the cell 4 observation that the GT has more detail.

Cell 6: Image loader and side-by-side viewer

What the code does

What the code does

- `load_image_pair(name)` reads the input and GT with OpenCV and converts BGR to RGB. OpenCV loads images as BGR, so skipping the conversion would swap red and blue and make every color judgment wrong.
- `plot_image_pairs(names)` shows, for each sample, the input, the GT, and a heatmap of `|GT – Input|` with the mean difference in the title.
- It is called on `0.jpg`, `10.jpg`, `25.jpg`, and `100.jpg`.

Output: a 4×3 grid of images (input, GT, difference heatmap).

Why we do it: looking at the data is the most important sanity check. It shows what the "task" actually is: what the network must change to get from input to GT. The heatmap shows where and how much the GT differs from the input.

What the result should tell us: raw images show blue or green casts and low contrast, and GTs look color-corrected and clearer. A bright, diffuse heatmap means the correction is global (color and lighting), while concentrated bright areas mean local corrections. The helper functions are reused for the rest of the notebook.

Cell 7: Degradation archetypes

What the code does

- `compute_color_bias` returns, for an image, the mean R, G, B, the brightness (average of the three), a green bias (`G / (R+B)`), and a blue bias (`B / (R+G)`).
- Applies it to 500 random inputs and picks the extreme image for each category: highest green bias, highest blue bias, lowest brightness, and highest brightness.
- Plots the four with `plot_image_pairs`.

Output: green archetype `732.jpg`, blue `3421.jpg`, dark `4159.jpg`, and "hazy" `1905.jpg`.

Why we do it: the dataset is not one problem but several, namely color cast, low light, and haze. Extreme examples show the range the model must handle. The archetypes are reused in cells 8 and 10.

What the result should tell us: each archetype needs a different correction, for example removing a color cast versus brightening versus removing haze. A model that does well on average could still fail on one archetype, so these are good failure-case test images later.

Caveat: "hazy" is just the brightest image, which is a rough proxy. A very bright image is not necessarily hazy, so treat that label with caution. The 500-image sample also means the extremes come from a subset, not all 4,279.

Cell 8: Per-channel histograms

What the code does: for one image (the green archetype, `732.jpg`), shows the input and GT images, and below them overlaid R, G, B histograms computed with `cv2.calcHist`. The legend shows each channel's mean.

Output: a 2×2 figure with images on top and the histograms below.

Why we do it: a color cast is a statement about channel distributions. Histograms show the exact shape of the problem, which means how compressed each channel is and where it sits, so they link a visual impression to a measurable one.

What the result should tell us: in the input, the red histogram is pushed toward dark values while green dominates. In the GT, the three channels are more similar and spread across a wider range. This is the "red channel loss" that motivates channel-aware designs like WaterMamba's CCOSS module.

Cell 9: Mean channel intensity over 300 images

What the code does: samples 300 pairs, computes the mean R, G, B for input and GT, averages them across the sample, shows a table, and draws a grouped bar chart.

Output

Channel	Raw input	Enhanced GT
Red	57.37	96.57
Green	121.04	107.89
Blue	107.85	107.53

Why we do it: cell 8 looked at one image. This cell shows the pattern holds across the dataset, so it is a statistical statement and not an anecdote.

What the result tells us

- Raw images have very weak red (57) compared with green (121) and blue (108). This confirms the physics of underwater light, where red is absorbed first.
- The GT raises red by about 39 points, lowers green by about 13, and leaves blue almost unchanged. The GT channels end up nearly equal (97, 108, 108).
- Implication: the model must learn a channel-dependent correction, mostly boosting red. This is evidence for using channel attention or per-channel modeling, and for adding a color-consistency term to the loss.

Cell 10: Color-space decomposition (LAB and HSV)

What the code does: for the blue archetype (`3421.jpg`), converts input and GT to LAB and HSV and shows, for each, the image, L* (lightness), a* (green↔red axis), and HSV saturation.

Output: a 2×4 figure (input row, GT row).

Why we do it: RGB mixes brightness and color. LAB and HSV separate them, so we can ask which of lightness, color, or saturation changed. Methods such as Ucolor use several color spaces for exactly this reason.

What the result should tell us: if L* changes a lot, the GT mostly fixes illumination and contrast. If a* shifts toward red and saturation recovers, it also fixes color. Seeing both explains why a loss function based on RGB alone may under-weight color or contrast errors.

Cell 11: Sharpness and contrast

What the code does

- `compute_sharpness_and_contrast` : converts to grayscale, takes the variance of the Laplacian (an edge/focus measure, where higher means sharper), and the standard deviation of intensity (RMS contrast).
- Evaluates the same 300 pairs, draws boxplots (sharpness on a log scale), and prints the median gain, GT divided by input.

Output: median sharpness gain 5.10× and **median contrast gain 1.20×**.

Why we do it: this puts numbers on "how much better is the ground truth". It shows how much detail and contrast a model has to restore, and which of the two matters more.

What the result tells us

- Sharpness changes much more than contrast (5× vs 1.2×). Enhancement here is mainly about restoring edges and texture, which supports using spatial modeling such as the SOSS block.
- Caution: a 5× jump is large. Part of it may come from the GT being post-processed (sharpened, or saved at higher JPEG quality) and not purely from scene recovery. A model may not match it exactly, and that is expected.
- The log scale is needed because Laplacian variance spans orders of magnitude.

Cell 12: Baseline PSNR and SSIM (the "do nothing" score)

What the code does

- `calculate_psnr`: `20·log10(255/√MSE)` between two images.
- `calculate_ssim`: a hand-written SSIM with an 11×11 Gaussian window (σ = 1.5) on each channel, averaged over the channels.
- Scores the raw input against its GT for 100 pairs and plots histograms with the mean, min, and max in the titles.

Output: two histograms. The exact values appear only in the plot titles, so read them from the notebook.

Why we do it: this is the **floor**. If you did no enhancement and just returned the input, this is the score you would get. Any trained model must beat it, and a model's gain is measured relative to it. The papers' numbers (24-28 dB PSNR) only mean something compared with this baseline.

What the result should tell us

- A low baseline PSNR and SSIM means the inputs differ substantially from the GT, so there is a lot to learn.
- A wide spread between min and max means some images are much harder than others.
- The same function can later score model outputs, so check that it matches a standard library (such as `skimage`) before reporting results.

Cell 13: Train/validation/test split

What the code does

- `create_dataset_splits` shuffles the metadata with a fixed seed, takes the first 80% as train, the next 10% as validation, and the remainder as test.
- Saves the result, with a `split` column, to `lsui_splits.csv` in the dataset folder.

Output

Split	Images
Train	3,423
Validation	427
Test	429

Why we do it

- Train data fits the weights, validation data guides tuning and early stopping, and test data gives the final unbiased score. The test set must never be used for tuning.
- Saving the split to a file means every experiment uses exactly the same division, so results are comparable

Saving the split to a file means every experiment uses exactly the same division, so results are comparable.

What the result tells us: we have about 3.4k pairs for training, which is modest and suggests data augmentation (flips, random crops). The split is random. If you compare against published results, remember that the official LSUI split is different (4,500 train and 504 test).

Minor code note: `.loc[n_train:n_train+n_val]` includes both ends, so one row is briefly labeled "val" and then overwritten as "test". The final counts are right, but `.iloc` slicing would be cleaner.

Summary of findings

Finding	Evidence	Consequence for the project
Dataset is complete	4,279 / 4,279 pairs, 0 missing	Safe to train on all of it
Images are small and vary in size	Median about 320×256, max 1280×1024	Resize or crop to a fixed size (e.g. 256×256)
Strong red loss	Red mean 57 vs green 121 in inputs	Need channel-aware modeling and color-aware loss
GT balances the channels	GT means 97 / 108 / 108	Target is neutral, balanced color
GT is much sharper	5.1× median Laplacian gain	Spatial detail restoration matters
Contrast gain is modest	1.2×	Color and sharpness matter more than contrast
Mixed degradations	Green, blue, dark, and bright archetypes	Evaluate on each type, not only on averages
A baseline exists	Cell 12 PSNR/SSIM of raw input	Report model improvement over this floor

Suggested next steps

1. Build a PyTorch `Dataset` driven by `lsui_splits.csv`, with paired augmentation.
2. Check the notebook's SSIM against `skimage.metrics.structural_similarity`.
3. Record the baseline PSNR/SSIM numbers from cell 12 in this report once you have run it.
4. Train the model and compare its PSNR/SSIM against the baseline, reporting results per archetype where possible.